15-using-test-doubles-effectively/10/api_request_tracker/spec/api_request_tracker_spec.rb

```
RSpec.describe APIRequestTracker do
  let(:request) { Request.new(:get, '/users') }

  it 'increments the request counter' do
    expect_any_instance_of(MetricsReporter).to receive(:increment).with(
      'api.requests.get_users'
    )

    APIRequestTracker.new.process(request)
  end
end
```

Here, we're calling expect_any_instance_of in place of plain expect, with the class as an argument. (Similarly, allow has an allow_any_instance_of counterpart.) This technique does help us get our class under test. But it definitely has significant drawbacks.

First, this tool is a *very* blunt hammer. You have no fine-grained control over how individual instances behave. Every instance of the named class (and its subclasses) will be affected, including ones you might not know about inside third-party libraries. Going back to our laboratory metaphor, when you bring a solution to a boil for an experiment, you probably don't want to boil all the liquid in the building!

Second, there are a lot of edge cases. You might wonder, for instance, whether expect_any_instance_of(MetricsReporter).to receive(:increment).twice means one instance must receive both calls to increment, or whether two different instances can each receive one call. The answer happens to be the former, but future readers may assume the latter and misunderstand your spec. Subclassing is another situation where it's easy to get mixed up, particularly when the subclass overrides a method you've stubbed.

Finally, this technique tends to calcify existing, suboptimal designs, rather than supporting you in improving the design of your code.

Stubbing a Factory Method

One slightly less far-reaching technique is to stub a factory method—typically SomeClass.new—to return a test double instead of a normal instance. Here's what that looks like for our APIRequestTracker test:

15-using-test-doubles-effectively/11/api_request_tracker/spec/api_request_tracker_spec.rb

```
it 'increments the request counter' do
  reporter = instance_double(MetricsReporter)
  allow(MetricsReporter).to receive(:new).and_return(reporter)
  expect(reporter).to receive(:increment).with('api.requests.get_users')

  APIRequestTracker.new.process(request)
end
```